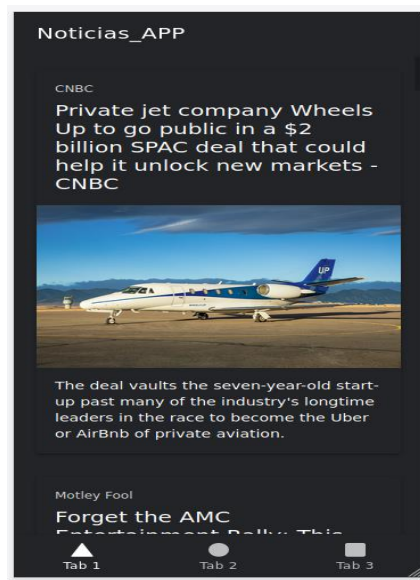


5.1 Servicios

Los servicios en ionic se comportan igual que en angular. En nuestro ejemplo, utilizaremos los servicios para crear una aplicación de noticias en la que veremos como:

1. Crear una aplicación real
2. Consumir APIS para desplegar información
3. Uso de componentes personalizados
4. Uso de componentes ionic
5. Uso de plugins
6. Almacenamiento en el storage del dispositivo

Y muchas otras cosas.



- Para crear un servicio ***ionic g s carpeta/nombre***
Si la carpeta no existe la crea
- Una vez creado el servicio, para poder utilizarlo tenemos que importar el HttpClient de angular/common/http en el appmodule, en la versión 18 de angular se incluye en el apartado providers provideHttpClient()

```
constructor(private http:HttpClient) { }

getOpcionesMenu(){
  return this.http.get<Iitem[]>('/assets/data/opciones.json');
}

ngOnInit() {
  this.lista=this.dataservice.getOpcionesPaginaMenu();
}
```

- Escribimos el servicio tal y como hacíamos en angular utilizando el método get del objeto HttpClient

```
http : HttpClient
http.get(URL o ruta)
```

Para utilizarlo, inyectamos el servicio en la clase correspondiente a la página donde vayamos a usar los datos que nos proporcionará el servicio.

Hay que recordar que el método get nos devuelve un objeto de tipo “Observable”, si lo utilizamos tal cual y queremos recorrerlo hay que añadir en el html el pipe | async

Si queremos transformar ese Observable en un array, en este caso no usaremos luego en el html el pipe, podemos usar el método subscribe, y como argumento de dicho método definimos una función que nos genere el array. El método subscribe lo usaremos en el componente para evitar problemas con la asincronía. Si de todas formas el objeto se “intenta pintar” en html antes de ser recibido, debemos usar *ngIf .

Ejemplo: Aplicación en formato tabs que mostrará las noticias obtenidas, de un api gratuito, en tiempo real. Tendrá tres tabs:

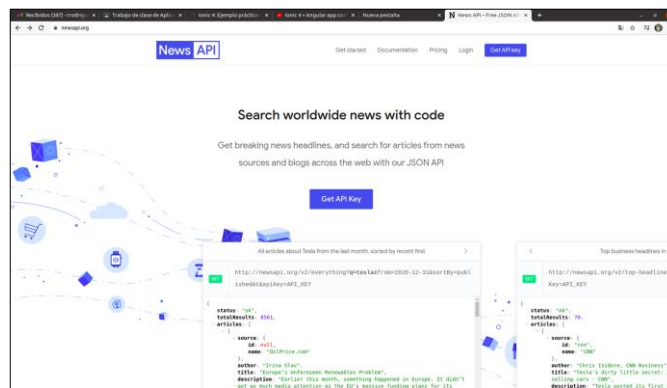
1. Todas las noticias se cargan en la primera

2. En la segunda mostraremos las categorías que nos ofrecen y mostraremos las de la categoría seleccionada
3. Tendremos la posibilidad de marcar noticias como favoritas y las mostraremos en la tercera

Veamos cómo hacer el primer punto

PASO 1 : registro en la API NewsApi.org

Nos tenemos que registrar en el API de noticias para poderlo utilizar

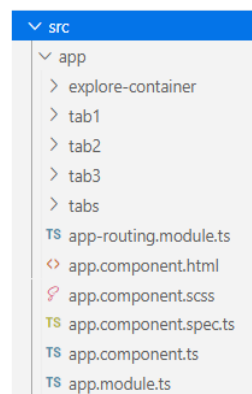


A través de Get Api Key nos registramos en este servicio. Obtenemos nuestra key.

PASO 2: Creamos el proyecto

Creamos proyecto **ionic tab ionic start ruta/nombre tabs**

Se han generado 5 carpetas dentro de src , container, contiene un componente que se va a páginas de los tabs



explorer-
utilizar en las

tab1, tab2 y tab3 son las páginas que contienen lo que queremos que se muestre en cada pestaña. Tabs es la página maestra donde vamos a mostrar los tabs con los iconos y controlar la ruta para que se muestre el contenido de la página correspondiente al tab activo.

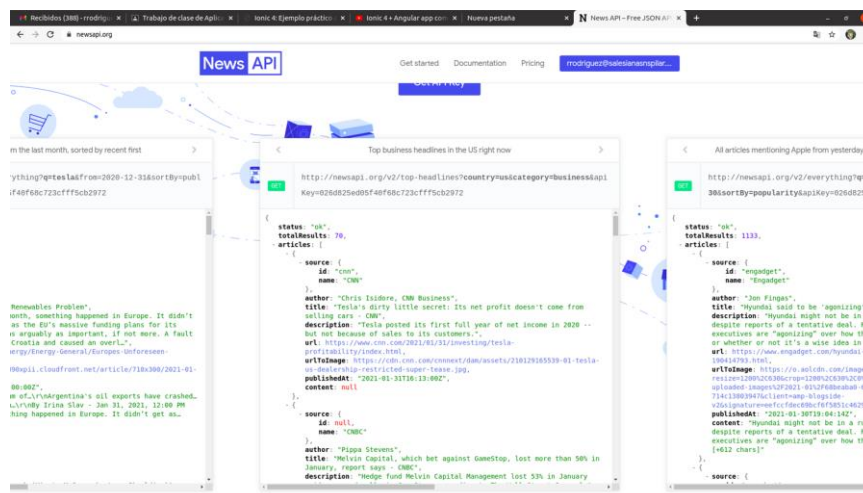
Vamos a crear una carpeta pages, para mover esas otras carpetas y que no estén en la raíz.

PASO 3: Copiamos nuestro api key en el archivo environments.ts

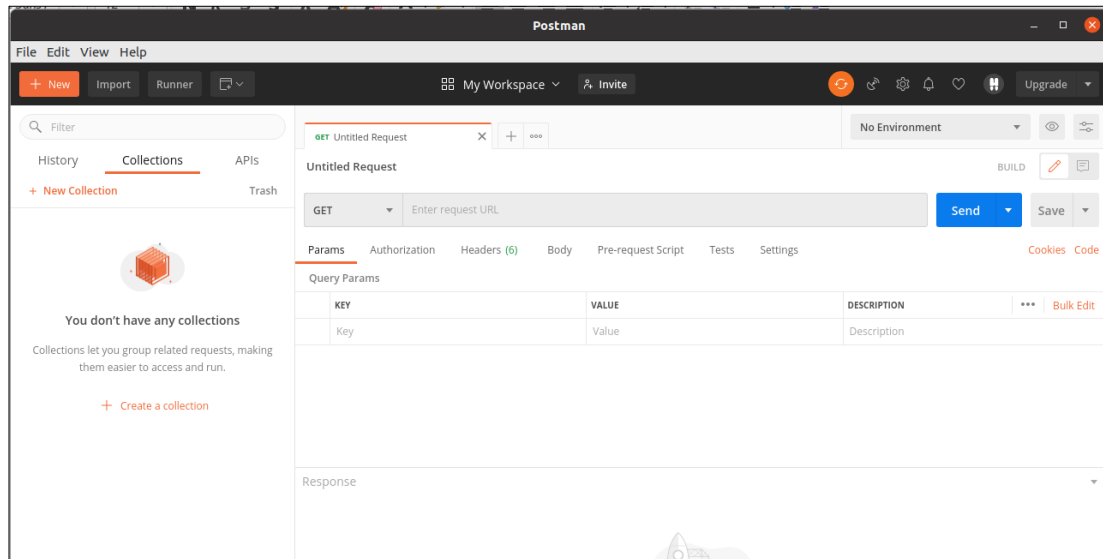
```
src > environments > TS environment.ts > [🔗] environment > [🔑] apikey  
1 // This file can be replaced during build by using the  
2 // `ng build --prod` replaces `environment.ts` with  
3 // The list of file replacements can be found in `angular  
4  
5 export const environment = {  
6   production: false,  
7   apikey: '026d825ed05f40f68c723cfff5cb2972'  
8 };  
9
```

A la variable podemos darle el nombre que queramos. También lo añadimos en el archivo environment.product.ts para que no se nos olvide al poner en producción la app.

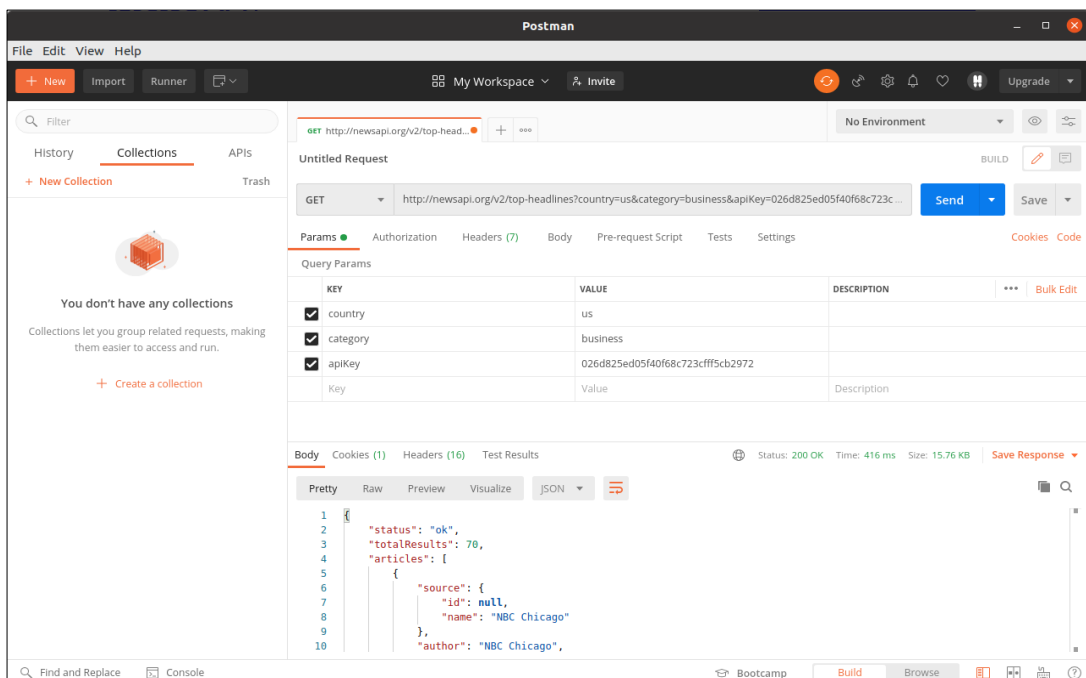
Vamos a la web de NewsApi.org y vemos unos ejemplos, nos fijamos en el segundo



Ahí aparece ya nuestra apiKey. Arrancamos post para comprobar que esa url funciona.



Enviamos la petición y vemos que efectivamente funciona



PASO 4: Crearemos nuestro servicio para consumir desde esa URL con el comando **ionic g s services/noticias**

Tenemos que importar el módulo `HttpClientModule` de `angular/common/http` en el `appmodule`

```
import {HttpClientModule} from '@angular/common/http';
```

Escribimos nuestro servicio tal y como lo hacíamos en angular

```
app > services > TS noticias.service.ts > NoticiasService > getTopHeadLines
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';

@Injectable({
  providedIn: 'root'
})
export class NoticiasService {

  constructor(private http: HttpClient) { }

  getTopHeadLines() {
    return this.http.get('http://newsapi.org/v2/top-headlines?country=us&category=business&apiKey=026d825ed05f40f68c723cfff5cb2972');
  }
}
```

Para probar que funciona, lo inyectamos en cualquier tab

```
src > app > pages > tab1 > TS tab1.page.ts > Tab1Page
1 import { Component, OnInit } from '@angular/core';
2 import { from } from 'rxjs';
3 import { NoticiasService } from '../services/noticias.service';
4 import { Article, RespuestaToHeadLines } from '../interfaces/interfaces';
5 import { Attribute } from '@angular/compiler/src/core';
6
7 @Component({
8   selector: 'app-tab1',
9   templateUrl: 'tab1.page.html',
10  styleUrls: ['tab1.page.scss']
11 })
12 export class Tab1Page implements OnInit {
13
14   constructor(private noticiasServ: NoticiasService) { }
15
16
17   ngOnInit() {
18     this.noticiasServ.getTopHeadLines().subscribe( respuesta => {console.log('noticias',respuesta)});
19   }
20
21 }
22
23
24
25
```

PASO 5: Interfaces para definir tipo de datos

Para mostrar los datos en pantalla nos vamos a crear un modelo para esos datos y no declararlos de tipo any. Añadimos una nueva carpeta interfaces y dentro de ella un archivo interfaces.ts

Nos vamos a postman y copiamos en el clipboard los datos

En el archivo interfaces.ts pulsamos CTRL + SHIFT + P se nos abre una caja para buscar la extensión. La seleccionamos y automáticamente se nos escriben las interfaces correspondientes al json. Es necesario tener esa extensión instalada en VS y en algunos casos instalar también el xclip

```
src > app > interfaces > TS interfaces.ts > Source
1  interface RootObject {
2      status: string;
3      totalResults: number;
4      articles: Article[];
5  }
6
7  interface Article {
8      source: Source;
9      author?: string;
10     title: string;
11     description?: string;
12     url: string;
13     urlToImage?: string;
14     publishedAt: string;
15     content?: string;
16 }
17
18 interface Source {
19     id?: string;
20     name: string;
21 }
```

Vamos a cambiar el nombre de la primera, la llamaremos RespuestaToHeadLines y exportamos las interfaces

```
src > app > interfaces > TS interfaces.ts > Source > name
1  export interface RespuestaToHeadLines {
2      status: string;
3      totalResults: number;
4      articles: Article[];
5  }
6
7  export interface Article {
8      source: Source;
9      author?: string;
10     title: string;
11     description?: string;
12     url: string;
13     urlToImage?: string;
14     publishedAt: string;
15     content?: string;
16 }
17
18 export interface Source {
19     id?: string;
20     name: string;
21 }
```

En el servicio específico el tipo de los datos que me proporcionará

```
getTopHeadLines(){
    return this.http.get<RespuestaToHeadLines>('http://newsapi.org/v2/top-headlines?country=us&category=business&apiKey=026d825ed05f40f68c723cfff5cb2972');
}
```

PASO 6: Visualizando los datos

En la clase de tab1 vamos a declarar un array para almacenar todos los artículos que nos proporciona la llamada al servicio

```
noticias: Article[]=[];
```


En el método subscribe ahora ya no quiero ejecutar la función console.log sino que quiero obtener todos los artículos del array que me retorna el servicio e ir uno a uno insertando en mi array noticias, para ello necesito el operador de javascript ...

```
this.noticiasServ.getTopHeadLines().subscribe( respuesta => { this.noticias.push(
...respuesta.articles) });
```

La clase me quedaría

```
src > app > pages > tab1 > ts tab1.page.ts > Tab1Page
1  import { Component, OnInit } from '@angular/core';
2  import { from } from 'rxjs';
3  import { NoticiasService } from '../services/noticias.service';
4  import { Article, RespuestaToHeadLines } from '../interfaces/interfaces';
5  import { Attribute } from '@angular/compiler/src/core';
6
7  @Component({
8    selector: 'app-tab1',
9    templateUrl: 'tab1.page.html',
10   styleUrls: ['tab1.page.scss']
11 })
12 export class Tab1Page implements OnInit {
13
14   noticias: Article[]=[];
15   constructor(private noticiasServ: NoticiasService) {}
16
17   ngOnInit(){
18     this.noticiasServ.getTopHeadLines().subscribe( respuesta => { this.noticias.push( ...respuesta.articles) });
19   }
20 }
21
22
23
24
```

Ahora ya podemos continuar con el html de tab1 para recorrer el array noticias

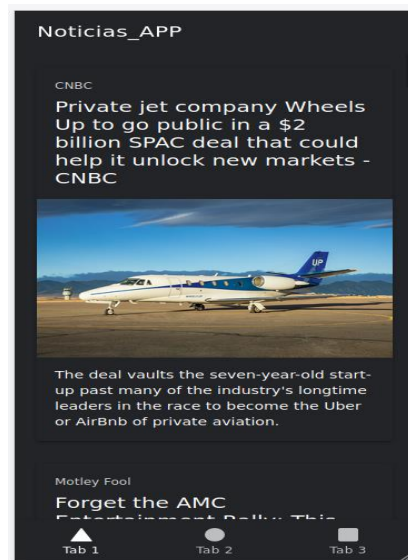
Hagamos los cambios, vamos a añadir varios componentes de ionic: ion-grid, ion-card, ion-image (ver su funcionamiento en la documentación oficial). Nos quedaría:

```

src > app > pages > tab1 > tab1.page.html > ion-header:ion-no-border > ion-toolbar > ion-title
1  <ion-header class="ion-no-border">
2    <ion-toolbar color="dark">
3      <ion-title>
4        Noticias_APP
5      </ion-title>
6    </ion-toolbar>
7  </ion-header>
8
9  <ion-content color="dark">
10
11    <ion-grid fixed>
12      <ion-row>
13        <ion-col size="12" size-lg="3" size-md="4" size-sm="6" size-xs="12" *ngFor="let n of noticias">
14          <ion-card color="dark">
15            <ion-card-header>
16              <ion-card-subtitle><span class="subtitulo">{{n.source.name}}</span></ion-card-subtitle>
17              <ion-card-title>{{n.title}}</ion-card-title>
18            </ion-card-header>
19
20            <ion-img *ngIf="n.urlToImage" [src]="n.urlToImage"></ion-img>
21
22            <ion-card-content>
23              <p>{{n.description}}</p>
24            </ion-card-content>
25          </ion-card>
26        </ion-col>
27      </ion-row>
28    </ion-grid>
29  </ion-content>
30
31
32
33

```

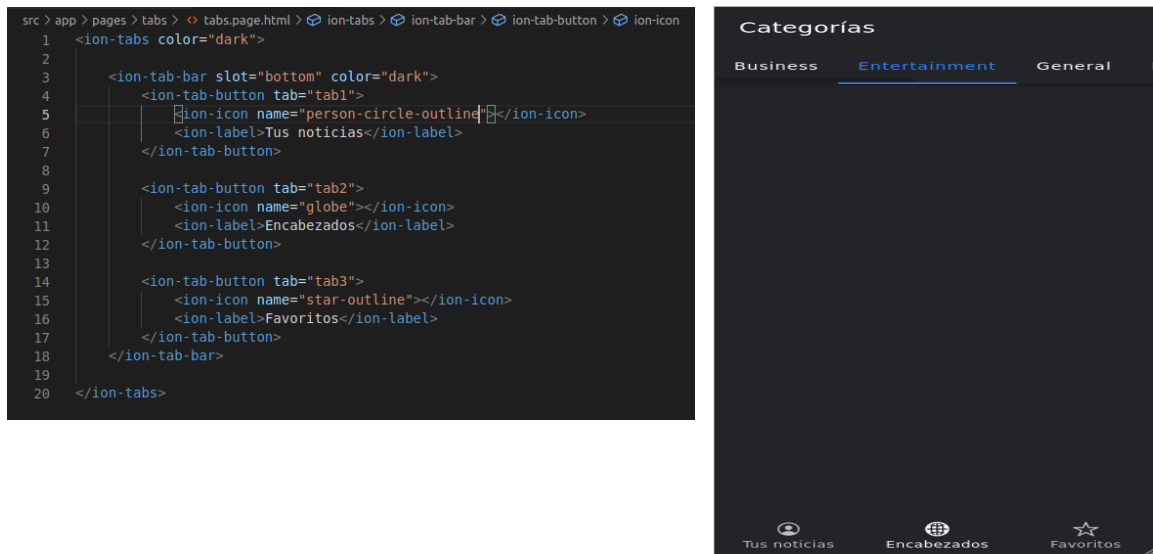
Y la ejecución de la aplicación



En el punto 2, queremos en Tab2 mostrar las categorías de noticias que trabaja este API y poder seleccionar una para que sólo se visualicen las correspondientes a dicha categoría

PASO 1: Visualizando los datos

Empezamos cambiando el archivo tabs.html para que en cada tab aparezcan un título e icono adecuados



Vamos a diseñar la pestaña tab2 para que se vea igual que en la imagen de arriba, incluiremos un ion-segment para poner todas las categorías de noticias que nos ofrece newsapi.org, para ello declaramos un array en la clase y copiamos los valores que obtenemos de la web

En la documentación de la web, elijo Top Headlines. Nos muestra la información sobre el servicio elegido. Entre los parámetros del request se encuentra category. Esas son las categorías que copiamos en nuestro array.

category The category you want to get headlines for. Possible options: **business**, **entertainment**, **general**, **health**, **science**, **sports**, **technology**. Note: you can't mix this param with the **sources** param.

Por tanto la clase quedaría

```
src > app > pages > tab2 > TS tab2.page.ts > Tab2Page > constructor
1  import { Component } from '@angular/core';
2
3  @Component({
4    selector: 'app-tab2',
5    templateUrl: 'tab2.page.html',
6    styleUrls: ['tab2.page.scss']
7  })
8  export class Tab2Page {
9
10   categorias : string[] = ['Business', 'Entertainment', 'General', 'Health', 'Science', 'Sports', 'Technology'];
11
12   constructor() {}
13
14 }
15
```

Diseñemos ahora la plantilla html

```
src > app > pages > tab2 > tab2.page.html > ion-content
1  <ion-header>
2    <ion-toolbar color="dark">
3      <ion-title>
4        Categorías
5      </ion-title>
6    </ion-toolbar>
7    <ion-segment scrollable mode="md">
8      <ion-segment-button mode="md" *ngFor="let categoria of categorias " [value]="categoria ">
9        <ion-label class="ion-text-capitalize">{{ categoria }}</ion-label>
10      </ion-segment-button>
11    </ion-segment>
12  </ion-header>
13
14  <ion-content color="dark ">
15
16  </ion-content>
```

Vamos a añadir algo de css al archivo de este tab2

```
src > app > pages > tab2 > tab2.page.scss > ion-segment-button checked
1  ion-segment-button {
2    background-color: var(--ion-color-dark) !important;
3    color: white !important;
4  }
5
6  .segment-button-checked {
7    color: var(--ion-color-primary) !important;
8  }
```

Implementamos el método en el servicio que nos proporcione las noticias según la categoría elegida. Será igual que el anterior, pero recibiendo como parámetro una categoría.

```
const url=environment.apiUrl + '/top-headlines?country=us';

@Injectable({
  providedIn: 'root'
})
export class NoticiasService {

  constructor(private http: HttpClient) { }

  getTopHeadLines(){
    return this.http.get<RespuestaToHeadLines>(url+'apiKey='+environment.apikey);
  }

  getTopHeadLinesCategory(cat:string){
    return this.http.get<RespuestaToHeadLines>([url+'category='+ cat+'&apiKey='+environment.apikey]);
  }
}
```

Dicha categoría la recibimos de la clase tab2

```
export class Tab2Page implements OnInit {

  categorias = ['Business', 'Entertainment', 'General', 'Health', 'Science', 'Sports','Technology'];
  noticias: Article[]=[];

  constructor(private noticiasServ: NoticiasService) {}

  ngOnInit(){
    this.cargarNoticias(this.categorias[0]);
    //Para que se carguen nada más entrar en tab2 la primera categoría que es la que por defecto estará seleccionada
  }

  noticiasPorCategoría(event){
    this.noticias=[]; //Para que cuándo cambio de categoría se limpie el array pq si no añadirá
    this.cargarNoticias([event.detail.value]);
  }

  cargarNoticias(cat:string){
    this.noticiasServ.getTopHeadLinesCategory(cat).subscribe(
      respuesta=>{ this.noticias.push(...respuesta.articles);}
    )
  }
}
```

Al tener el componente noticias ya implementado, el html de tab2 no varía apenas en la llamada a dicho componente

```

<ion-header>
  <ion-toolbar color="dark">
    <ion-title>
      Categorías
    </ion-title>
  </ion-toolbar>
  <ion-segment scrollable mode="md" (ionChange)=noticiasPorCategoria($event) [value]="categorias[0]">
    <ion-segment-button mode="md" *ngFor="let categoria of categorias " [value]="categoria ">
      <ion-label class="ion-text-capitalize">{{ categoria }}</ion-label>
    </ion-segment-button>
  </ion-segment>
</ion-header>

<ion-content color="dark ">
  <app-noticias [noticias]="noticias"></app-noticias>
</ion-content>

```

Por último, vamos a dar la posibilidad de poder compartir en redes sociales las noticias que elijamos

Añadiremos a cada noticia un botón que nos muestre un action-sheet que tendrá acciones para compartir en redes, añadir a favoritos...

Compartir en redes sociales

1. Crear ese “botón” en cada noticia

Añadimos el componente `i-button-icon-only` en el html del componente noticia. El icono será

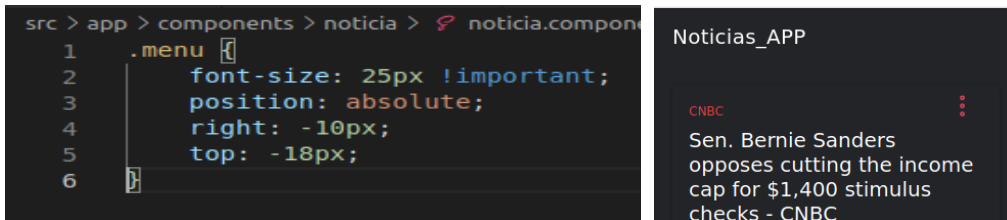
⋮

Para colocarlo a la derecha del subtítulo y que se vea tendremos que aplicar nuestro propio CSS

```

src > app > components > noticia > noticia.component.html > ion-card > ion-card-header > ion-card-subtitle > div.menu
1 <ion-card color="dark">
2   <ion-card-header>
3     <ion-card-subtitle><span class="subtitulo">{{n.source.name}}</span>
4     <div class="menu">
5       <ion-button (click)="onClick()" fill="clear" color="danger">
6         <ion-icon slot="icon-only" name="ellipsis-vertical-outline"></ion-icon>
7       </ion-button>
8     </div>
9   </ion-card-subtitle>
10   <ion-card-title>{{n.title}}</ion-card-title>
11 </ion-card-header>
12
13   <ion-img *ngIf="n.urlToImage" [src]="n.urlToImage"></ion-img>
14
15   <ion-card-content>
16     <p>{{n.description}}</p>
17   </ion-card-content>
18 </ion-card>

```



2. Implementar el onClick

Igual que hicimos en el tema en que explicamos este componente, copiamos dentro del evento onClick el código de uso de la documentación, añadimos la variable al constructor y podemos darle estilos css a nuestro gusto

```
@Input() n: Article;
constructor(public actionSheetController: ActionSheetController) { }

ngOnInit() {}

async onClick(){
  const actionSheet = await this.actionSheetController.create({
    cssClass: 'my-custom-class',
    buttons: [ {
      text: 'Share',
      icon: 'share',
      handler: () => {
        console.log('Share clicked');
      }
    }, {
      text: 'Favorite',
      icon: 'heart',
      handler: () => {
        console.log('Favorite clicked');
      }
    }, {
      text: 'Cancel',
      icon: 'close',
      role: 'cancel',
      handler: () => {
        console.log('Cancel clicked');
      }
    }
  ])
  await actionSheet.present();
}
```

Si probamos la aplicación veremos que funciona (yo no voy a retocar css)